

Name :- Kartik Ram Birajdar

Roll No :- A – 13

Branch :- AI&DS

Assignment-9

1. What is a Data Science Pipeline? Explain its key stages with examples.

A Data Science Pipeline is a structured sequence of steps used to collect, process, analyze, and model data to extract useful insights or make predictions. Key Stages:

1 Data Collection

Gathering data from sources like databases, APIs, CSV files, etc. Example: Collecting student marks from a CSV file.

Data Cleaning (Preprocessing)

Handling missing values, removing duplicates, fixing errors. Example: Filling missing marks with average values.

Exploratory Data Analysis (EDA)

Understanding data using statistics and visualization. Example: Plotting histogram of marks.

Feature Engineering

Creating new features from existing data. Example: Total marks → average marks.

Feature Scaling

Normalizing or standardizing data for better model performance. Example: Scaling marks between 0–1. Model Building Applying machine learning algorithms. Example: Using linear regression to predict marks.

Model Evaluation

Checking accuracy using metrics.

Example: Calculating accuracy or RMSE.

2. What is Exploratory Data Analysis (EDA)?

EDA (Exploratory Data Analysis) is the process of analyzing and summarizing datasets using statistics and visualization techniques to understand patterns and relationships.

Types of Analysis:

a) Univariate Analysis

Involves analyzing a single variable. Helps understand distribution, central tendency. Example:

Mean, median, mode Histogram of age **b) Bivariate Analysis**

Involves two variables. Helps identify relationships between variables.

Example:

- Scatter plot of height vs weight
- Correlation between marks and study hours

Feature	Label Encoding	One-Hot Encoding
Definition	Converts categories into numbers	Creates binary columns
Example	Red=1, Blue=2	Red=[1,0], Blue=[0,1]
Data Type	Single column	Multiple columns
Use Case	Ordinal data	Nominal data

When to Use:

Label Encoding:

When categories have order Example: Low < Medium < High **One-Hot**

Encoding:

When no order exists Example: Colors (Red, Blue, Green)

4. Why is Feature Scaling Required? Feature scaling ensures all features have equal importance and improves model performance.

Why Needed:

Algorithms like KNN, SVM depend on distance Prevents large values from dominating

Feature	Normalization	Standardization
Range	0 to 1	No fixed range
Use	Neural Networks Most ML models Less	Sensitive to Outliers Yes

5. What is Feature Engineering?

Feature Engineering is the process of creating new features or modifying existing ones to improve model performance.

Why Important:

- Improves accuracy
- Helps model learn better patterns **Examples:**
 - Extracting day/month/year from date
 - Creating BMI = weight / height²
 - Combining features like total = math + science **How it**

Improves Performance:

- Provides more meaningful input to model
- Reduces noise
- Highlights hidden relationships

```
# Step 1: Import Libraries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classif

# Step 2: Load Dataset (Seaborn built-in dataset)
df = sns.load_dataset('titanic')

# Step 3: Basic Inspection
print(df.head())
print(df.info())
print(df.isnull().sum())

# Step 4: Data Preprocessing

# Select useful columns
df = df[['survived', 'pclass', 'sex', 'age', 'fare', 'embarked']]

# Handle missing values
imputer = SimpleImputer(strategy='mean')
df['age'] = imputer.fit_transform(df[['age']])

df['embarked'].fillna(df['embarked'].mode()[0], inplace=True)

# Encode categorical variables
le = LabelEncoder()
df['sex'] = le.fit_transform(df['sex'])
df['embarked'] = le.fit_transform(df['embarked'])

# Step 5: Exploratory Data Analysis (EDA)

# Survival count
sns.countplot(x='survived', data=df)
plt.title("Survival Count")
plt.show()

# Age distribution
```

```
sns.histplot(df['age'], kde=True)
plt.title("Age Distribution")
plt.show()

# Step 6: Feature Scaling
scaler = StandardScaler()
df[['age', 'fare']] = scaler.fit_transform(df[['age', 'fare']])

# Step 7: Split Data
X = df.drop('survived', axis=1)
y = df['survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 8: Model Building
model = LogisticRegression()
model.fit(X_train, y_train)

# Step 9: Prediction
y_pred = model.predict(X_test)

# Step 10: Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_p
```

```

      survived  pclass      sex  age  sibsp  parch      fare embarked  cl
0           0       3    male  22.0     1     0    7.2500         S   Th
1           1       1  female  38.0     1     0   71.2833         C   Fi
2           1       3  female  26.0     0     0    7.9250         S   Th
3           1       1  female  35.0     1     0   53.1000         S
      Fi4      0      3    male  35.0     0     0    8.0500
      S   Th

```

```

      who  adult_male deck  embark_town alive  alone  0
man      True  NaN  Southampton    no  False
1  woman      False   C   Cherbourg   yes  False
2  woman      False  NaN  Southampton   yes  True
3  woman      False   C   Southampton   yes  False
4  man      True  NaN  Southampton    no  True

```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 891 entries, 0 to 890
```

```
Data columns (total 15 columns):
```

```

#      Column      Non-Null Count  Dtype
---  -
0  survived      891 non-null      int64
1  pclass        891 non-null      int64
2  sex           891 non-null      object
3  age           714 non-null      float64
4  sibsp         891 non-null      int64
5  parch         891 non-null      int64
6  fare          891 non-null      float64
7  embarked      889 non-null      object
8  class         891 non-null      category
9  who           891 non-null      object
10 adult_male    891 non-null      bool
11 deck         203 non-null      category
12 embark_town  889 non-null      object
13 alive        891 non-null      object  14  alone      891 non-
      null      bool

```

```
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
```

```
memory usage: 80.7+ KB
```

```
None survived
```

```
0 pclass
```

```
0 sex
```

```
0 age
```

```
177 sibsp
```

```
0 parch
```

```
0 fare
```

```
0 embarked
```

```
2 class
```

```
0 who
```

```
0 adult_male
```

```
0 deck
```

```
688 embark_town
```

```
2 alive
```

```
0 alone
```

```
0 dtype: int64
```

```
/tmp/ipykernel_6486/3761747590.py:30: FutureWarning: A value is tryin
The behavior will change in pandas 3.0. This inplace method will neve
```

```
For example, when doing 'df[col].method(value, inplace=True)', try us
```

```
df['embarked'].fillna(df['embarked'].mode()[0], inplace=True)
```



